1 CandyGrab Continued

Today we get to bring the CandyGrab agents from lecture to life! Download and unzip:

- https://www.cs.cmu.edu/~15281/recitations/rec1/candygrab.zip

This code is meant for you to mess around with during recitation. Don't worry about breaking things, as you can always redownload the original code. Inside the CandyGrab directory you'll find:

- `candygrab.py`: The game environment
- `agent.py`: The Agent base class. All agents inherit from this simple base class.
- `agent*.py`: Implementations of various agents. We saw some of these in lecture.
- `play_one.py`: Simple script to have two agents play one round of the game
- `play_many.py`: Simple script to have two agents play many rounds of the game

Feel free to edit any of these files or make copies of `play_*.py` or `agent*.py` to create your own battles or agents. But first, we'll have you just run and inspect the code to get an idea of how things are working in this simple game. You can start by running the following:

- `python36 play_one.py`
- `python36 play_many.py`

Take a look through the source code and answer the following questions:

(a) How is the state represented in the code?

The number of remaining pieces

(b) What is Agent005's strategy?

Choose a random action

(c) What is Agent003's strategy?

Copy the opponent's action

(d) How do the AgentRL stats change when playing against an Agent008 rather than an Agent007? What difference in the two agents do you think causes this? (You should modify `play_many.py`)

With an Agent008, fewer of the table entries have N/A's. This is because Agent008 will act randomly when neither of its actions is optimal (it can't force its opponent to have mod 3 pieces). Because of this randomness, the RL agent will see more game states than it might have versus an Agent007.

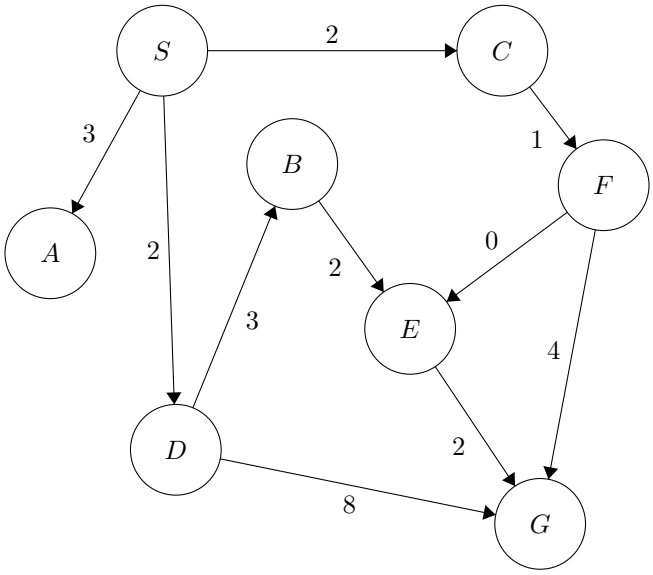
(e) When we run the games 1000 and 10000 times, how do the AgentRL stats change when playing against another AgentRL? Which states seem to be good states to take? (You should modify `play_many.py`)

The stats do not change that much since the two agents are both RL agents under explore mode (random actions), and neither agent is exploiting their table of statistics to try to win more games. For example, looking at AgentRL's table of statistics, action 2 from state 5 has a much higher win probability (75%) than action 1 from state 5 (35%). However, because explore mode defaults to true, AgentRL will continue choosing random actions from state 5, rather than the more attractive action 2.

(Bonus) AgentRL has an `explore_mode` setting that defaults to `True`. What happens when you play 500 games, turn off explore mode (`bb8.explore_mode = False`), and then play 500 more games?

The `explore_mode` indicates whether or not the agent should explore - act randomly, so that it can record as many different game states into its internal table as possible, or exploit - where it uses the information it has learned to play as well as it can. Since `explore_mode` is initially true, the RL agent is only acting randomly. However, once it uses its table, it begins to do much better, and win a lot more of its games. This is a concept we'll come back to when we learn about reinforcement learning (RL) :)

2 Search Algorithms



Using each of the following graph search algorithms presented in lecture, write out the order in which nodes are added to the explored set, with start state S and goal state G. Break ties in alphabetical order. Additionally, what is the path returned by each algorithm? What is the total cost of each path?

Note: For BFS, DFS, and IDS, the answers may vary depending on the order in which you add successor nodes (children) onto the frontier.
For instance, on (b) DFS we could have pushed S's children in the order S-A, S-C, S-D, thus popping off S-D first and resulting in the returned path S-D-G.
Since this order is arbitrary, in a homework or test situation, we would specify the order to do this in (or accept any valid answers). If you're unsure about your particular answer, feel free to ask at OH or on Piazza!

(a) Breadth-first

Explored set: S, A, C, D, F, B
Frontier: ~~∅~~, ~~S-A~~, ~~S-C~~, ~~S-D~~, ~~S-C-F~~, ~~S-D-B~~, S-D-G, S-C-F-E
Path: S-D-G
Path cost: 10

(b) Depth-first

Explored set: S, A, C, F, E
Frontier: ~~∅~~, S-D, ~~S-C~~, ~~S-A~~, ~~S-C-F~~, S-C-F-G, ~~S-C-F-E~~
Note: S-C-F-E-G is never added to the frontier because G is already on the frontier.
Path: S-C-F-G
Path cost: 7

(c) Uniform cost

Explored set: S, C, D, A, F, E, B
Frontier: ~~(S, 0)~~, ~~(S-A, 3)~~, ~~(S-C, 2)~~, ~~(S-D, 2)~~, ~~(S-C-F, 3)~~, ~~(S-D-B, 5)~~, ~~(S-D-G, 10)~~, ~~(S-C-F-E, 3)~~, ~~(S-C-F-G, 7)~~, (S-C-F-E-G, 5)
Note: Horizontal strike-through means the node was replaced on the frontier. Specifically, ~~(S-D-G, 10)~~ was replaced by ~~(S-C-F-G, 7)~~, which was in turn replaced by (S-C-F-E-G, 5)
Path: S-C-F-E-G
Path cost: 5

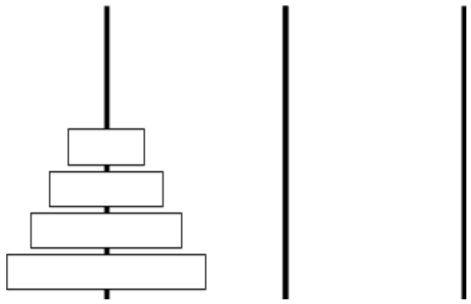
(d) Iterative deepening

depth = 0:
Explored set: S
Frontier: ~~S~~

depth = 1:
Explored set: S, A, C, D
Frontier: ~~S~~, ~~S-D~~, ~~S-C~~, ~~S-A~~

depth = 2:
Explored set: S, A, C, F, D, B
Frontier: ~~S~~, ~~S-D~~, ~~S-C~~, ~~S-A~~, ~~S-C-F~~, ~~S-D-B~~, S-D-G
Path: S-D-G
Path cost: 10

3 Tower of Hanoi



The Tower of Hanoi is a canonical puzzle studying problem solving and formulation. The puzzle starts with n disks of different sizes stacked in order of size (see picture above) on a peg, along with two empty pegs. We can move disks freely between the pegs, but larger disks cannot be stacked on top of smaller ones. The goal is to move all disks to the third peg.

We will attempt to formulate Tower of Hanoi as a search problem.

(a) How could we represent this puzzle as a problem, ie. what would the states be?

Possible answer: store 3 lists tracking the disks stacked on each peg. Enumerate the disks 1 to n.

For the following questions, assume that you have used your state representation in your answer above.

(b) What is the size of the state space in terms of n?

Assume that a disk number can appear on any of the three lists, independently of where the other numbers are. This means that there are 3^n possible combinations of the three lists. We don't need to factor in the order of those lists, because the only legal order is sorted from smallest to largest. For example, with only two disks, the number of combinations is 3^2. Specifically:

- `([1, 2], [], [])`
- `([1], [2], [])`
- `([1], [], [2])`
- `([2], [1], [])`
- `([], [1, 2], [])`
- `([], [1], [2])`
- `([2], [], [1])`
- `([], [2], [1])`
- `([], [], [1, 2])`

(c) What is the starting state?

`([1, 2, ..., n], [], [])`

(d) From some given state, what legal actions are there?

We can pop the first integer from any list and push it to the front of any other list, given that this integer is less than the current first integer of the target list (or that the target list is empty).

(e) What is the goal test? Remember that this determines whether a given state is a goal state, `goalTest(state)`.

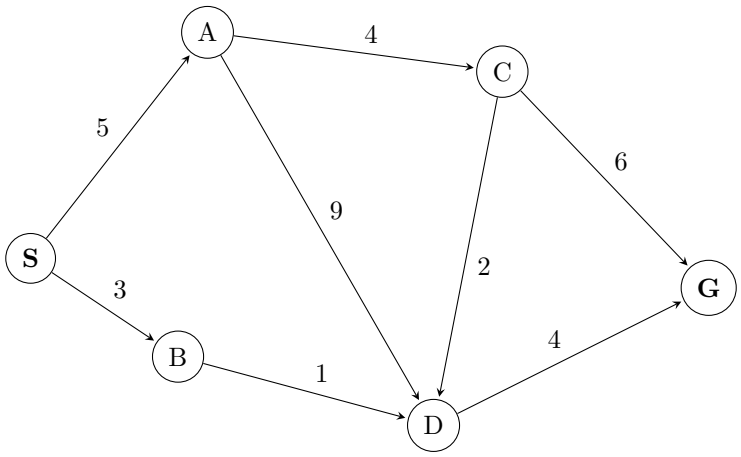
`goalTest` should check that `state == ([], [], [1, 2, ..., n])`.

1 Designing & Understanding Heuristics

Today, we will be taking a closer look at how the performance of A^* is affected by the heuristics it uses. To do this, we'll be using the graph below. You may have noticed that no heuristic values have been provided (*Recall:* What is A^* without heuristic values?). This is because we'll be working in pairs to come up with heuristics ourselves!

Please find someone next to you to work with, and decide between yourselves who will design an admissible heuristic and who will design a consistent heuristic. Then, *independently* create these heuristics for the given graph by annotating each node with a heuristic value.

When you have completed your heuristic, exchange papers with your partner, and work together to answer the questions below.



(a) Write down the path found by running A^* using your heuristic on the graph above.

This will depend on the provided heuristic. Recall that A^* expands the node in its frontier that has the lowest $f(n) = g(n) + h(n)$ value.

(b) Work with your partner to come up with a heuristic that's admissible but not consistent.

Heuristics will vary from team to team, and there may be many correct solutions. Feel free to come to OH or post on Piazza if you're unsure about a particular answer.

(c) (Bonus) Explain why a consistent heuristic must also be admissible. You may assume that the heuristic value at a goal node is always 0.

Recall the definitions of admissibility and consistency.

A heuristic is admissible if it never overestimates the true cost to a nearest goal.
A heuristic is consistent if, when going from neighboring nodes a to b, the heuristic difference/step cost never overestimates the actual step cost. This can also be re-expressed as the triangle inequality mentioned in Lecture 3.

Let $h(n)$ be the heuristic value at node n and $c(n, n+1)$ be the cost from node n to $n+1$. We're given the assumption that $h(g) = 0$. Informally, we want to backtrack from the goal node, showing that if we start with an admissible node (which $h(g) = 0$ is by default), its parent nodes will be admissible

through the rule of consistency, and this pattern continues.

Consider some node n and assume the heuristic value at n is admissible. We want to show that any of its parent nodes, say $n-1$, will also be admissible by the definition of consistency.

By consistency, we get that:

$$h(n-1) - h(n) \leq c(n-1, n)$$
$$= h(n-1) \leq c(n-1, n) + h(n)$$

$c(n-1, n)$ is the actual path cost from $n-1$ to n. Since we know $h(n)$ is admissible, we know that $h(n)$ has to be less than or equal to the path cost from n to goal node g.

Thus, $h(n-1) \leq$ the path cost from ($n-1$ to n) + $h(n)$.
Since $h(n)$ is less than or equal to path cost from (n to g),
$h(n-1) \leq$ path cost from ($n-1$ to g).

This by definition is admissible.

Formal proof (for students interested):
Assume we have some consistent heuristic h. Also assume $h(g) = 0$, where g is a goal node. By definition of consistency, $h(n) \leq c(n, n+1) + h(n+1)$ for all nodes n in the graph. We want to show that for all n, $h(n) \leq h^*(n)$ (the definition of admissibility) also holds.

Base Case: We begin by considering the $g-1$th node in any path where g denotes the goal state.

$$h(g-1) \leq c(g-1, g) + h(g) \tag{1}$$

Because g is the goal state, by assumption, $h(g) = h^*(g)$. Therefore, we can rewrite the above as

$$h(g-1) \leq c(g-1, g) + h^*(g)$$

and given that $c(g-1, g) + h^*(g) = h^*(g-1)$, we can see:

$$h(g-1) \leq h^*(g-1)$$

as desired.

Inductive Hypothesis: Assume that for some arbitrary node (which lies on a path from start to goal) $g-k$ that $h(g-k) \leq h^*(g-k)$.

Inductive Step: To see if this is always the case, we consider the $g-k-1$th node in any of the paths we considered above (e.g. where there is precisely one node between it and the goal state). The cost to get from this node to the goal state can be written as

$$h(g-k-1) \leq c(g-k-1, g-k) + h(g-k)$$

From our base case above, we know that

$$h(g-k-1) \leq c(g-k-1, g-k) + h(g-k) \leq c(g-k-1, g-k) + h^*(g-k)$$

$$h(g-k-1) \leq c(g-k-1, g-k) + h^*(g-k)$$

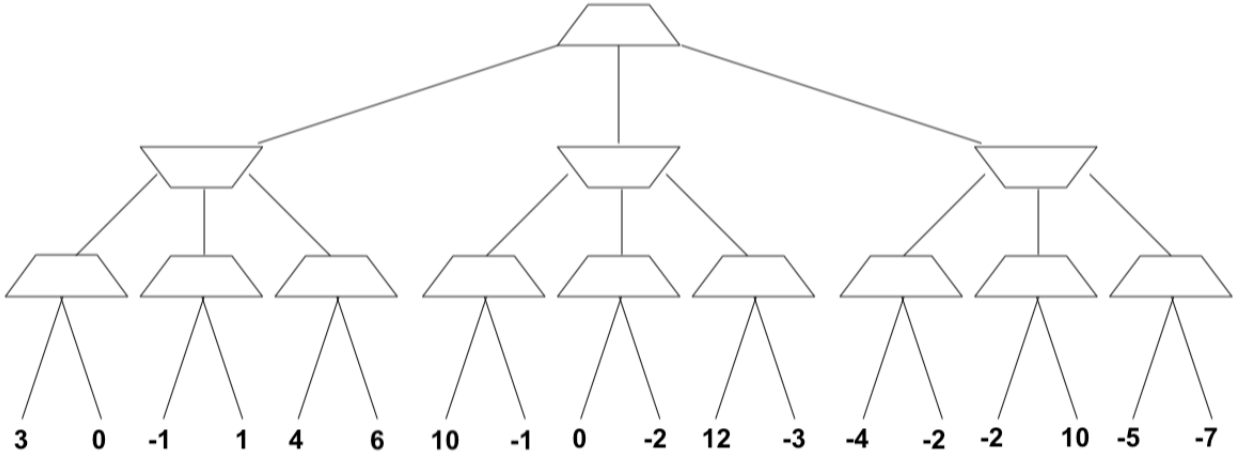
And again, we know that $c(g-k-1, g-k) + h^*(g-k) = h^*(g-k-1)$, so we can see:

$$h(g-k-1) \leq h^*(g-k-1)$$

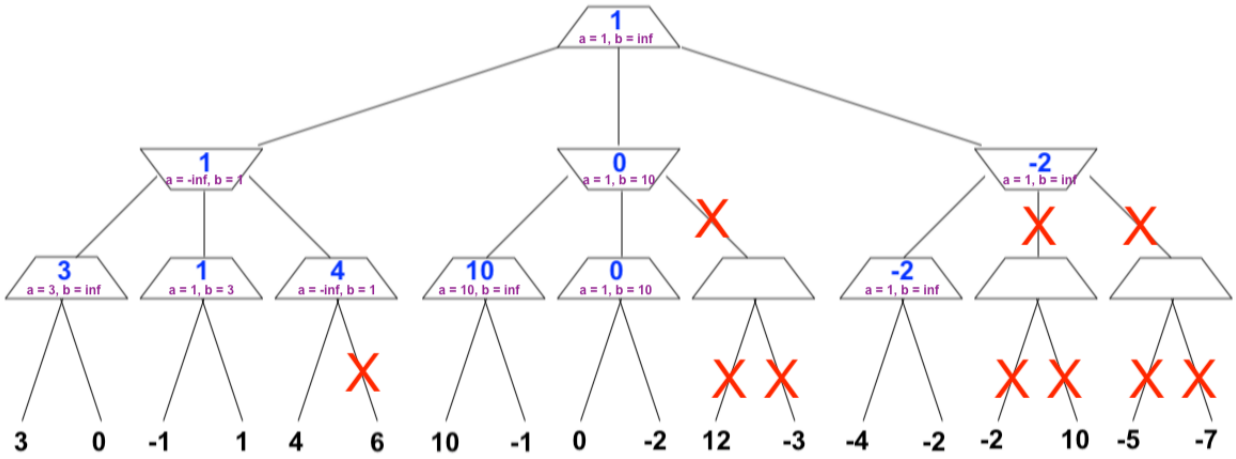
By the inductive hypothesis, this holds for all nodes, proving that consistency does imply admissibility!

2 Adversarial Search

Consider the following game tree, where the root node is a maximizer. Using alpha beta pruning and visiting successors from left to right, record the values of alpha and beta at each node. Furthermore, write the value being returned at each node inside the trapezoid. Put an 'X' through the edges that are pruned off.



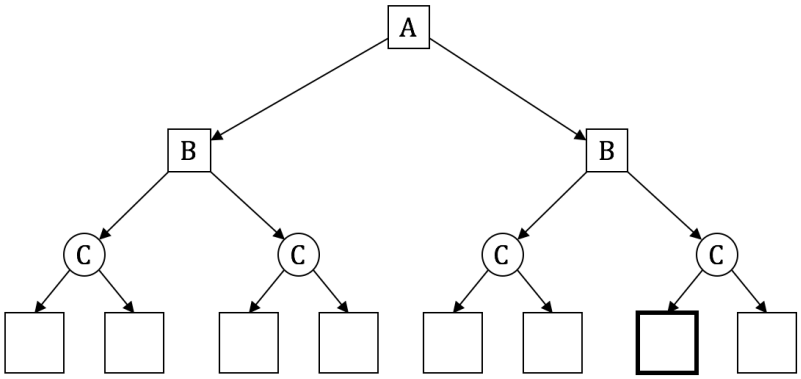
The following picture shows the final α and β (denoted a and b) values at each node.

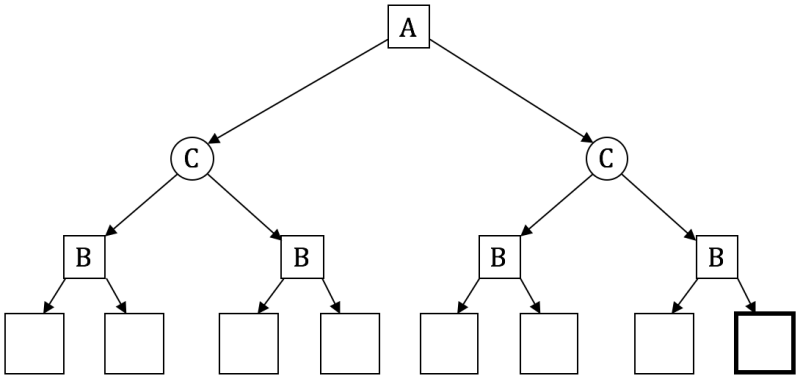


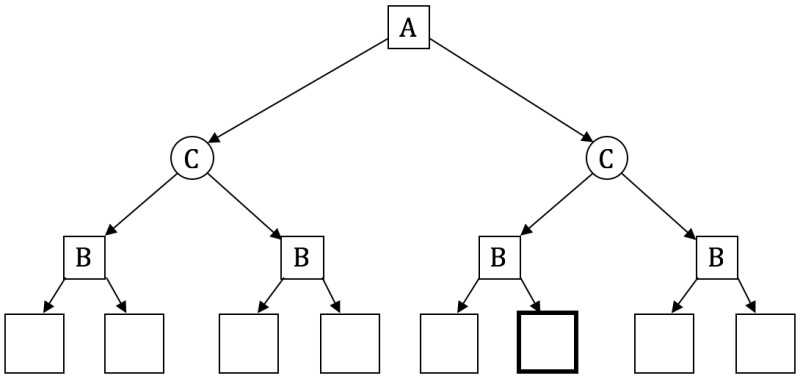
3 Alpha Beta Expectimax

In this question, player A is a minimizer, player B is a maximizer, and C represents a chance node. All children of a chance node are equally likely. Consider a game tree with players A, B, and C. In lecture, we considered how to prune a minimax game tree - in this question, you will consider how to prune an expectimax game tree (like a minimax game tree but with chance nodes). Assume that the children of a node are visited left to right.

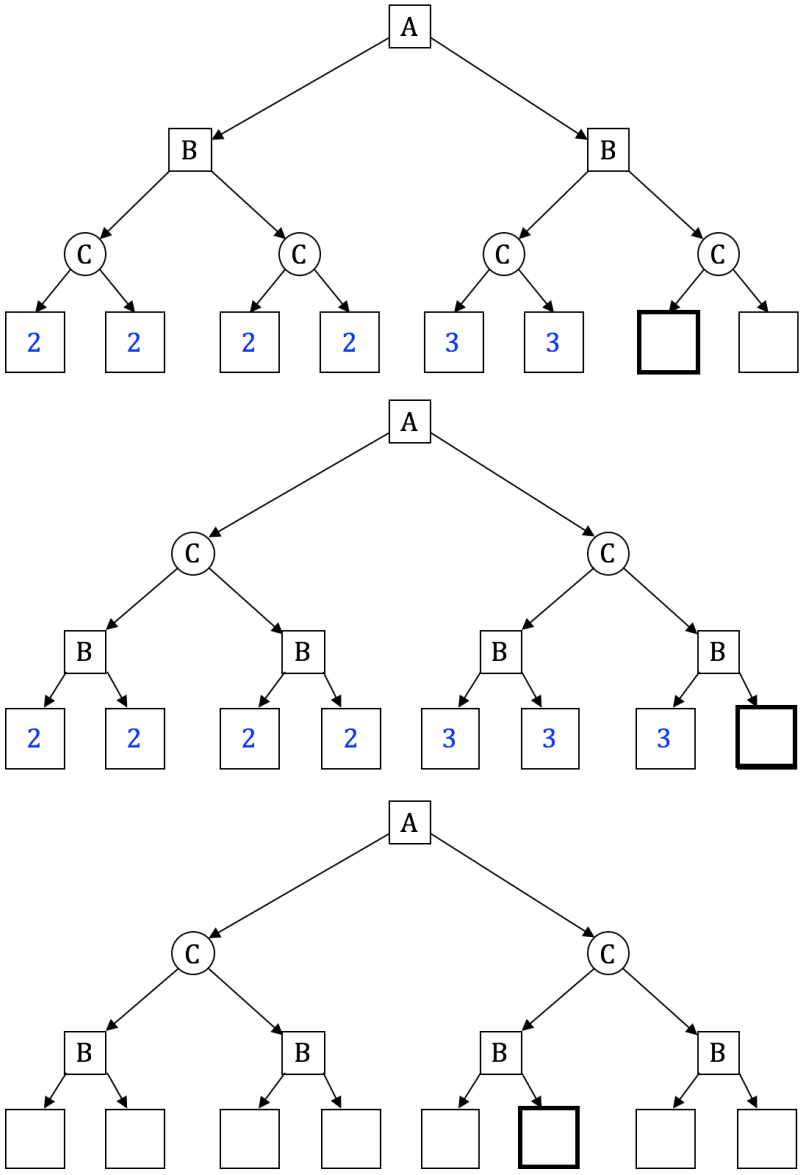
For each of the following game trees, give an assignment of terminal values to the leaf nodes such that the bolded node can be pruned (it doesn't matter if you prune more nodes), or write "not possible" if no such assignment exists. You may give an assignment where an ancestor of the bolded node is pruned (since then the bolded node will never be visited). You should not prune on equality, and your terminal values must be finite.







Answers may vary.



Not possible. At the bolded node, the minimizer can guarantee a score of at most the value of its left subtree. However, since we have not visited all the children of C, there is no bound on the value that C could attain. So, we need to continue exploring nodes until we can put a bound on C's value, which means we must explore the bolded node.

4 True/False Section

For each of the following questions, answer true or false and provide a brief explanation (or counterexample, if applicable).

(a) Depth-first search always expands at least as many nodes as A^* search with an admissible heuristic.

False: If the minimum goal path consists of d nodes, a lucky DFS might expand exactly those d nodes to reach the goal. A^* could potentially expand some other nodes before finding that path (those nodes would have a lower $f(n)$ value than the final goal's).

(b) Assume that a rook can move on a chessboard any number of squares in a straight line, vertically or horizontally, but cannot jump over other pieces. Manhattan distance is an admissible heuristic for the problem of moving the rook from square A to square B in the smallest number of moves.

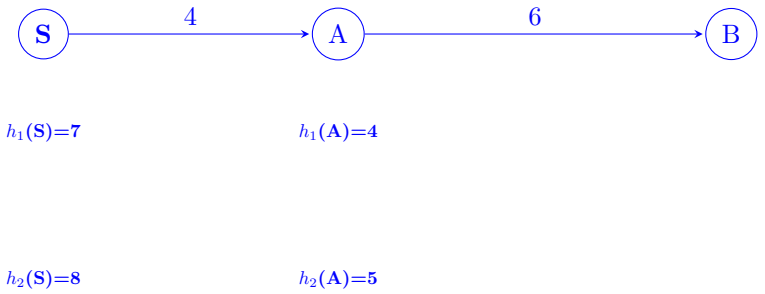
False: A rook can move from one corner to the opposite corner across a 4x4 board in two moves, although the Manhattan distance from start to finish is 6.

(c) Euclidean distance is an admissible heuristic for Pacman path-planning problems.

True: Euclidean distance is the minimum distance to travel between any two points. Thus, it will always be less than or equal to Pacman's (who only moves horizontally or vertically) actual cost to travel along some path, making it admissible ($0 \leq h(n) \leq h^*(n)$).

(d) The sum of several admissible heuristics is still an admissible heuristic.

False:



Both of these heuristics (h_1 and h_2) are admissible, but if we sum them, we find that $h3(s) = 15$ and $h3(a) = 9$. However, this is not admissible.

For (e) and (f), consider an adversarial game tree where the root node is a maximizer, and the minimax value of the game (i.e., the value of the root node after running minimax search on the game tree) is V_M. Now, also consider an otherwise identical tree where every minimizer node is replaced with a chance node (with an arbitrary but known probability distribution). The expectimax value of the modified game tree is V_E.

(e) V_M is guaranteed to be less than or equal to V_E.

True: The maximizer is guaranteed to be able to achieve v_M if the minimizer acts optimally. They can potentially do better if the minimizer acts suboptimally (e.g. by acting randomly). The expectation at chance nodes can be no less than the minimum of the nodes' successors.

(f) Using the optimal minimax policy in the game corresponding to the modified (chance) game tree is guaranteed to result in a payoff of at least V_E.

False: In order to achieve v_E in the modified (chance) game, the maximizer may need to change their strategy. The minimax strategy may avoid actions where the minimum value is less than the expectation. Moreover, even if the maximizer followed the expectimax strategy, they are only guaranteed v_E *in expectation*.

1 Discussion Questions

(a) What is the difference between Forward Checking and AC-3?

Forward checking and AC-3 both enforce arc consistency, but forward checking is more limited. Whenever a variable X is assigned, forward checking enforces arc consistency only for arcs that are pointing to X, which will reduce the domains of the neighboring variables in the constraint graph. Forward checking stops at this point, but AC-3 will continue to enforce arc consistency on neighboring arcs until there are no more variables whose domain can be reduced. As a result, FC ensures arc consistency of the assigned variable and its neighbors only, while AC-3 ensures arc consistency for the whole graph.

(b) Why does a tree-structured CSP take $O(nd^2)$ to solve?

If the constraint graph has no loops, the CSP can be solved in $O(nd^2)$ time compared to general CSPs, where worst-case time is $O(d^n)$. To solve a tree-structured CSP, first pick any variable to be the root of the tree, and choose an ordering of the variables such that each variable appears after its parent in the tree. Such an ordering is called a topological sort. Any tree with n nodes has $n-1$ arcs, so we can make this graph directed arc-consistent in $O(n)$ steps, each of which must compare up to d possible domain values for two variables, for a total time of $O(nd^2)$.

(c) Why would one use the following heuristics for CSP?

(i) Minimum Remaining Values (MRV)

MRV: "Which variable should we assign next?"

- Fail fast
- We have to assign all variables at some point, so we might as well do hard stuff first (allowing us to prune the search tree faster/realize we need to backtrack)

(ii) Least Constraining Value (LCV)

LCV: "Which value should we try next?"

- We just want one solution.
- We don't try all combinations of value, so we should try ones that are likely to lead to a solution.

2 CSP: Air Traffic Control

We have five planes: A, B, C, D, and E and two runways: international and domestic. We would like to schedule a time slot and runway for each aircraft to either land or take off. We have four time slots: 1, 2, 3, 4 for each runway, during which we can schedule a landing or take off of a plane. We must find an assignment that meets the following constraints:

- Plane B has lost an engine and must land in time slot 1.
- Plane D can only arrive at the airport to land during or after time slot 3.
- Plane A is running low on fuel but can last until at most time slot 2.
- Plane D must land before plane C takes off, because some passengers must transfer from D to C.
- No two aircrafts can reserve the same time slot for the same runway.

(a) Complete the formulation of this problem as a CSP in terms of variables, domains, and constraints (both unary and binary). Constraints should be expressed implicitly using mathematical or logical notation rather than with words. Make sure to specify variables, domains, and constraints.

Variables: A, B, C, D, E for each plane.
Domains: a tuple (*runway type, time slot*) for runway type $\in$ {international, domestic} and time slot $\in \{1, 2, 3, 4\}$.
Constraints:

$$B[1] = 1$$

$$D[1] \geq 3$$

$$A[1] \leq 2$$

$$D[1] < C[1]$$

$$A \neq B \neq C \neq D \neq E$$

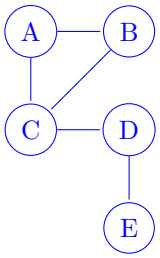
Note here we use B[1] to denote the second value of the tuple assigned to variable B. the time slot value, which is a number in $\{1, 2, 3, 4\}$

For the following parts, we add the following two constraints:

- Planes A, B, and C cater to international flights and can only use the international runway.
- Planes D and E cater to domestic flights and can only use the domestic runway.

(b) The addition of the two constraints above alters the CSP. Specifically, the domain does not need to include the runway type since this information is carried by the variable, and the binary constraints have changed. Determine the new domain and complete the constraint graph for this problem given the original constraints and the two added ones.

Variables: A, B, C, D, E for each plane.
Domain: $\{1, 2, 3, 4\}$
Constraint Graph:



Explanation of Constraints Graph: We can now encode the runway information into the identity of the variable, since each runway has more than enough time slots for the planes it serves. We represent the non-colliding time slot constraint as a binary constraint between the planes that use the same runways.

(c) What are the domains of the variables after enforcing arc consistency? Begin by enforcing unary constraints. (Cross out values that are no longer in the domain.)

Enforcing arc consistency with AC-3, we have the following domain as a result:

A	~~1~~	2	~~3~~	~~4~~
B	1	~~2~~	~~3~~	~~4~~
C	~~1~~	~~2~~	~~3~~	4
D	~~1~~	~~2~~	3	~~4~~
E	1	2	~~3~~	4

(explanation of process below)
Enforcing unary constraints (in an arbitrary order) first,

1. We cross out 2, 3, 4 from B's domain, adding arcs A → B and C → B to the queue.
2. We cross out 3, 4 from A's domain, adding arcs B → A and C → A to the queue.
3. We cross out 1, 2 from D's domain, adding arcs C → D and E → D to the queue.

Enforcing A → B, we cross out 1 from A's domain; add arcs B → A and C → A to the queue.
Enforcing C → B, we cross out 1 from C's domain; add arcs A → C, B → C, and D → C to the queue.
Enforcing B → A, no domain changes are necessary (all values remaining in B's domain have a consistent corresponding value in A's domain); no arcs are added.
Enforcing C → A, we cross out 2 from C's domain; add arcs A → C, B → C, and D → C to the queue.
Enforcing C → D, we cross out 3 from C's domain; add arcs A → C, B → C, and D → C to the queue.
Enforcing E → D, no domain changes are necessary.
Enforcing B → A, no domain changes are necessary.
Enforcing C → A, no domain changes are necessary.
Enforcing A → C, no domain changes are necessary.
Enforcing B → C, no domain changes are necessary.
Enforcing D → C, we cross out 4 from D's domain (there is no c in C's domain such that $c > 4$); add arcs C → D and E → D to the queue.
Enforcing A → C, no domain changes are necessary.
Enforcing B → C, no domain changes are necessary.
Enforcing D → C, no domain changes are necessary.
Enforcing C → D, no domain changes are necessary.
Enforcing E → D, we cross out 3 from E's domain; add arc D → E to the queue.
Enforcing D → E, no domain changes are necessary.
(phew!)

Note: For a general binary CSP, to enforce arc consistency before assigning any variables, you should add all arcs to the initial queue. For this problem, it can be easily seen that if there are no unary constraints, all the arcs will be consistent before any variable is assigned a value. As a result, we can start with the unary constraints and add arcs only for the related variables after enforcing the unary constraints.

(d) Arc-consistency can be rather expensive to enforce, and we believe that we can obtain faster solutions using only forward-checking on our variable assignments. Using the Minimum Remaining Values heuristic, perform backtracking search on the graph, breaking ties by picking lower values and characters first. List the (variable, assignment) pairs in the order they occur (including the assignments that are reverted upon reaching a dead end). Enforce unary constraints before starting the search.

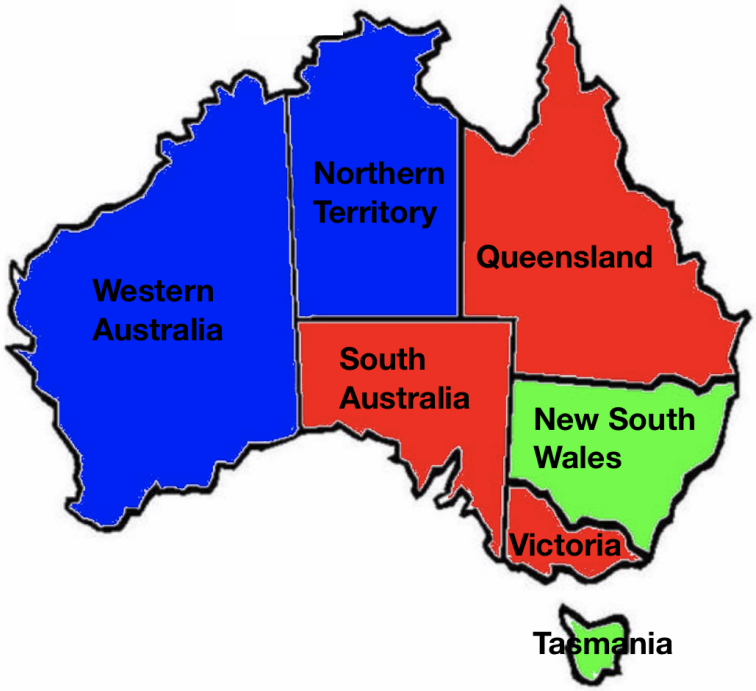
List of (variable, assignment) pairs:

(You don't have to use this table)

A	1	2	3	4
B	1	2	3	4
C	1	2	3	4
D	1	2	3	4
E	1	2	3	4

Answer: (B, 1), (A, 2), (C, 3), (C, 4), (D, 3), (E, 1)

3 Map Coloring with Local Search



Recall the various local search algorithms presented in lecture. Local search differs from previously discussed search methods in that it begins with a complete, potentially conflicting state and iteratively improves it by reassigning values. We will consider a simple map coloring problem, and will attempt to solve it with hill climbing.

(a) How is the map coloring problem defined (In other words, what are variables, domain and constraints of the problem)? How do you define states in this coloring problem?

- Variables: WA, NT, SA, Q, NSW, V, T (States in Australia)
- Domain: Green, Red, Blue
- Constraints: Adjacent countries can't have the same color assignment.
  e.g: Implicit: $WA \neq NT$
  Explicit: (WA, NT) $\in$ (red, blue), (red, green), (blue, red), (blue, green), (green, red), (green, blue)
- Problem state: a full coloring of the map (i.e., color assignments to all variables).

(b) Given a complete state (coloring), how could we define a neighboring state?

A neighboring state could be a full coloring of the graph with a different color assignment to only one variable.

(c) What could be a good heuristic be in this problem for local search? What is the initial value of this heuristic?

The heuristic could be the number of variable pairs that have conflicting colors. In the initial state, the following 3 pairs (WA-NT, Q-SA, SA-V) are conflicting, so the heuristic h = 3. (Note: there could be other possible heuristics for this problem.)

(d) Use hill climbing to find a solution based on the coloring provided in the graph.

Let h be our heuristic value.
In the original graph, we have 3 coloring conflicts as stated in (c). Depending on the search order, the assignment order might be different and the searched path lengths as well as coloring can also vary. We represent the coloring of states in a list with the following order: [WA, NT, Q, SA, NW, V, T]. Below are two examples of potential search paths.

- Step 1: h = 3. Conflicts are WA-NT, Q-SA, SA-V. We start with WA-NT. Coloring NT with Green would resolve the WA-NT conflict. Coloring: [B, G, R, R, G, R, G]
  Step 2: h = 2. Conflicts are Q-SA, SA-V. We can pick SA-Q pair and assign Blue to SA, which would resolve SA-Q and SA-V conflicts but will add coloring conflict for WA-SA pair. Still, it decreases the number of conflicts and is a better neighboring state. Coloring: [B, G, R, B, G, R, G]
  Step 3: h = 1. Conflict is just WA-SA pair. We can simply assign WA with Red to resolve this conflict, where we completed the search and found a solution to the problem. Coloring: [R, G, R, B, G, R, G]

We got pretty lucky in the search above and found a solution in 3 steps. However, local search may not always resolve conflicts optimally. Below is an example where it has to resolve the conflicts with more steps.

- Step 1: h = 3. Conflicts are WA-NT, Q-SA, SA-V. We start with WA-NT. Coloring WA with Green would resolve the WA-NT conflict. Coloring: [G, B, R, R, G, R, G]
  Step 2: h = 2. Conflicts are Q-SA, SA-V. We can resolve both of these conflicts by assigning Blue to SA, which will lead us to a better neighboring state with only one conflict: NT-SA. Coloring: [G, B, R, B, G, R, G]
  Step 3: h = 1. Conflicts are NT-SA. However, looking at all possible assignments to both of these two states, we see that no matter what color we assign to either one of them, we can't find a better neighboring state. Therefore the current iteration of hill climbing would end and we would restart from the initial state if we are applying random-restart hill climbing. An alternative is to use simulated annealing, which would allow us to sometimes move to states of higher heuristic value in order to escape local minima.

We see that in the second search, we need more steps to complete search. There are other possible search steps sequences depending on the choice on the order of conflicts to resolve, and the color assignment when resolving each conflict.

We can also use a genetic algorithm approach to find a solution to color the map. We could let the number of non-conflicting State pairs (in this case, WA-SA, NT-SA, Q-NT, SA-NW, Q-NW, NW-V) minus the number of conflicts in each state be our fitness function, and represent the variables in a list as above: [WA, NT, Q, SA, NW, V, T].

With a genetic algorithm, we first randomly select $2k$ full colorings of the graph with replacement (i.e. k pairs of parents), with probability proportional to their fitness function values. For each pair of parents, we would choose a cross over point $\in \{1...6\}$ to split the variable list in two halves and combine the color assignments with cross over to generate two offspring states.
(There are many other ways we could've implemented this crossover - this is just following the n-queens example from lecture.)

Then with a small probability, we could mutate the color assignments to the variables by one color, which means we would modify one of the values assigned to the variable list to a value in $\{Red, Green, Blue\}$. We would repeat the process until we find a good enough coloring that does not result in any conflicts within the map, or until it exceeds the max number of iterations.

(e) How is local search different from tree search?

Tree search keeps all unexplored alternatives on the frontier, where local search does not have a frontier but only keeps improving a single option until there's no better neighboring states.